

새싹 강동캠퍼스 베스핀글로벌

# AWS ALB + WAS Auto Scaling 수행평가

MISSION 3 | 기존 3-Tier/RDS 환경의 확장성 고도화

## Mission Focus

기존 VPC, WEB, RDS Multi-AZ 환경은 그대로 사용합니다. 이번 미션에서는 Internal ALB 와 Auto Scaling Group 을 WAS 계층에 추가하고, 의도적으로 가벼운 CPU 부하를 발생시켜 Scale-Out / Scale-In 과 트래픽 분산을 직접 확인합니다.

강사 젠아이랩스 박수현

권장 수행시간 2 시간

# 시나리오

## 고객사 시나리오

A 사는 기존 수행평가를 통해 WEB-Nginx, WAS-Flask, RDS Multi-AZ 로 구성된 3-Tier 서비스를 안정적으로 운영하고 있습니다. 최근 신규 상품 프로모션을 앞두고 특정 시간대에 요청이 급격히 증가할 가능성이 생겼습니다. 현재 WEB 서버는 WAS 서버의 Private IP 를 직접 바라보고 있어, WAS 서버를 추가하더라도 WEB 설정을 다시 변경해야 하는 문제가 있습니다.

고객사는 WAS 계층을 자동으로 확장할 수 있도록 개선하되, WEB 서버는 새로 생성되는 WAS 의 IP 를 알 필요가 없어야 한다고 요구했습니다. 또한 트래픽이 증가하면 WAS 가 자동으로 Scale-Out 되고, 부하가 사라지면 다시 Scale-In 되는 과정을 실제로 확인하고자 합니다.

여러분은 기존 환경을 재구축하지 않고 Internal Application Load Balancer 와 Auto Scaling Group 을 추가하여 확장 가능한 WAS 계층을 완성하고, Stress 서버에서 단계별 부하를 발생시켜 어느 WAS 서버가 요청을 처리하는지와 Auto Scaling 동작을 검증해야 합니다.

### 추가 운영 요구사항

또한 신규 애플리케이션 버전 배포 시 각 WAS 인스턴스에 개별 SSH 접속해 수정하지 않고, AMI 와 Launch Template 의 새 버전 및 Instance Refresh 를 이용하여 v1 에서 v2 로 일괄 교체할 수 있어야 합니다. 강사는 최종 웹 서비스의 /info 응답만으로 어떤 WAS 가 요청을 처리했는지와 배포 버전을 확인할 수 있어야 합니다.

## 이번 미션의 전제

- 기존 VPC 10.0.0.0/16, 6 개 Subnet, IGW/NAT/Route Table/Bastion 환경을 그대로 사용합니다.
- WEB-A / WEB-C 의 Nginx 와 RDS Multi-AZ 는 정상 동작하는 상태에서 시작합니다.
- WAS Subnet 은 AZ-A 10.0.11.0/24, AZ-C 10.0.21.0/24 를 그대로 사용합니다.
- 기존 상품 조회 기능과 RDS 연결 코드는 유지하고, Auto Scaling 실습에 필요한 최소 코드만 추가합니다.
- 기존 WAS 서버를 기존 이미지(AMI) 생성용으로 재사용합니다.

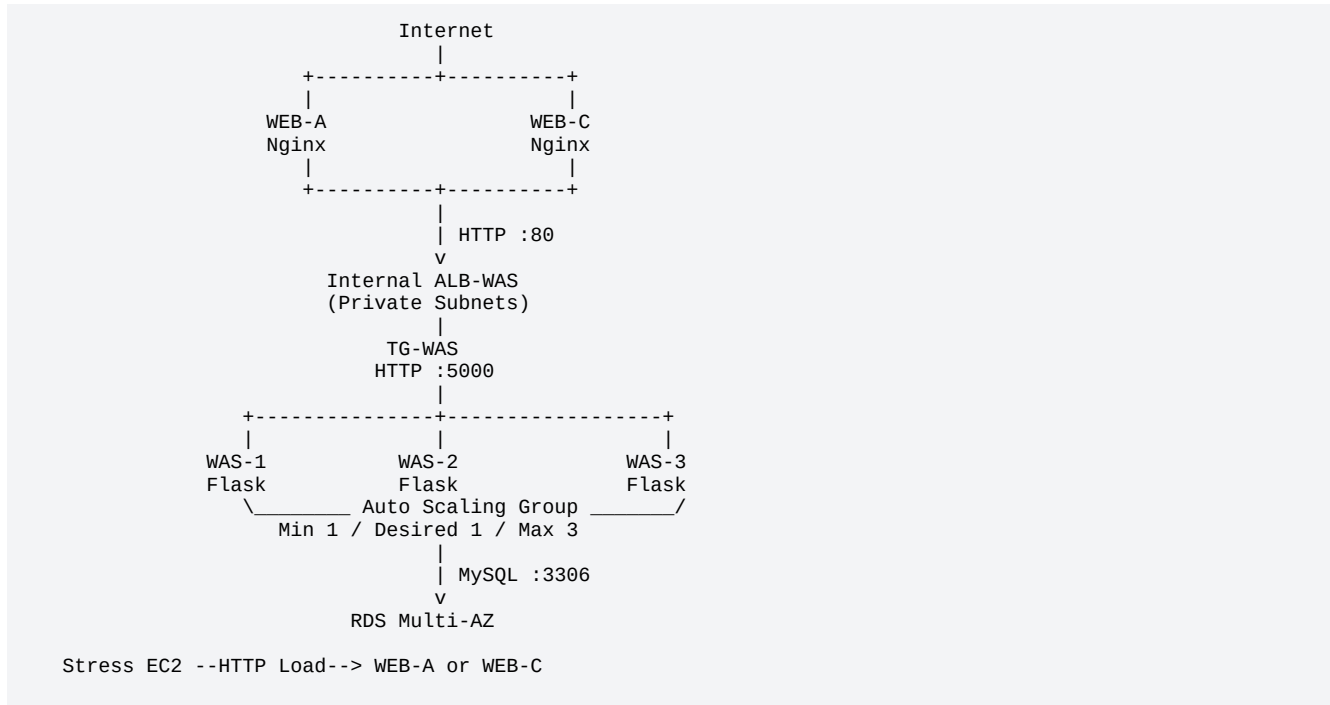
## Mission Goal

1. WEB 이 개별 WAS IP 가 아닌 Internal ALB DNS 로 접속하도록 변경한다.
2. WAS Auto Scaling Group 을 2 개 AZ 에 구성하고 Target Group 과 연결한다.
3. load=1~5 로 조절 가능한 /stress API 를 이용해 WAS CPU 부하를 발생시킨다.
4. WAS 가 1 대에서 최대 3 대까지 Scale-Out 되는 것을 확인한다.
5. 새 WAS 가 Target Group 에 자동 등록되고 실제 요청을 처리하는 것을 hostname 으로 확인한다.
6. 부하 제거 후 Scale-In 되는 것을 확인한다.

7. AMI v2 / Launch Template v2 / Instance Refresh 를 이용해 WAS 를 v1 에서 v2 로 교체하고 /info 에서 최종 배포 상태를 검증한다.

## 1. 목표 아키텍처

이번 미션의 핵심은 WEB 과 WAS 사이에 Internal ALB 를 넣어 WEB 이 개별 WAS 의 IP 를 알지 않아도 되게 만드는 것입니다.



### Before → After

| 구분           | 기존                   | Mission 3 이후                    |
|--------------|----------------------|---------------------------------|
| WEB 의 WAS 연결 | WAS Private IP 직접 지정 | Internal ALB DNS 만 지정           |
| WAS 수량       | 고정 EC2               | ASG 가 1~3 대 자동 조정               |
| 신규 WAS 등록    | 수동 구성 필요             | ASG → Target Group 자동 등록        |
| 트래픽 분산       | WEB 설정에 의존           | Internal ALB 가 Healthy WAS 로 분산 |

## 2. Step 1 - WAS 애플리케이션 준비

기존 Flask 상품 조회 코드는 유지합니다. 아래 코드만 추가하여 Health Check 와 단계별 CPU 부하, WAS hostname 확인 기능을 넣습니다.

### 2.1 import 추가

```
from flask import request
import socket
```

## 2.2 ALB Health Check 용 API

```
@app.route("/health")
def health():
    return "OK\n"
```

## 2.3 Auto Scaling 부하 테스트 API

```
@app.route("/stress")
def stress():
    load = int(request.args.get("load", 1))
    load = max(1, min(load, 5))

    result = 0
    for i in range(load * 100000):
        result += i * i

    return (f"WAS={socket.gethostname()} "
            f"VERSION={APP_VERSION} load={load}\n")
```

| load | CPU 계산 반복 횟수 | 용도                  |
|------|--------------|---------------------|
| 1    | 100,000      | 동작 확인               |
| 2    | 200,000      | 약한 부하               |
| 3    | 300,000      | 기본 Stress 권장        |
| 4    | 400,000      | 부하 강화               |
| 5    | 500,000      | Scale-Out 이 느릴 때 사용 |

## 2.4 상품 화면에 WAS + Version 표시

```
hostname = socket.gethostname()
html = f"<h2>WAS Server: {hostname} / Version: {APP_VERSION}</h2>"
html += "<h1>상품 목록</h1><ul>"
```

## 2.5 배포 상태 확인 API - /info

```
APP_VERSION = "v1"

@app.route("/info")
def info():
    return f"WAS={socket.gethostname()} VERSION={APP_VERSION}\n"
```

강사는 학생의 WEB Public IP 만 받아도 /info 를 반복 호출하여 ALB 분산 대상과 현재 배포 버전을 확인할 수 있습니다.

## 2.6 최종 app.py 예제

기존 RDS Endpoint, 계정, 비밀번호는 자신의 Mission 2 환경에 맞게 입력합니다. 아래 파일을 기준 WAS 에 저장한 뒤 AMI v1 을 생성합니다.

```
from flask import Flask, request
import pymysql
import socket
from werkzeug.serving import WSGIRequestHandler

app = Flask(__name__)
APP_VERSION = "v1"

class RealIPHandler(WSGIRequestHandler):
    def address_string(self):
        return self.headers.get("X-Real-IP") or super().address_string()

@app.route("/")
def index():
    conn = pymysql.connect(
        host="RDS_ENDPOINT",
        user="webuser",
        password="password",
        database="testdb"
    )

    cursor = conn.cursor()
    cursor.execute("SELECT * FROM products")
    products = cursor.fetchall()
    cursor.close()
    conn.close()

    hostname = socket.gethostname()
    html = f"<h2>WAS Server: {hostname} / Version: {APP_VERSION}</h2>"
    html += "<h1>상품 목록</h1><ul>"

    for product in products:
        html += f"<li>{product[0]} / {product[1]} / {product[2]}원</li>"

    html += "</ul>"
    return html

@app.route("/health")
def health():
    return "OK\n"

@app.route("/info")
def info():
    return f"WAS={socket.gethostname()} VERSION={APP_VERSION}\n"

@app.route("/stress")
def stress():
    load = int(request.args.get("load", 1))
    load = max(1, min(load, 5))

    result = 0
    for i in range(load * 100000):
        result += i * i

    return (f"WAS={socket.gethostname()} "
            f"VERSION={APP_VERSION} load={load}\n")

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000, request_handler=RealIPHandler)
```

### 3. Step 2 - Flask 자동 시작과 기준 AMI 생성

Auto Scaling 으로 새 EC2 가 생성되면 Flask 가 자동으로 실행되어야 합니다. 기준 WAS 에서 한 번만 systemd 서비스를 등록한 뒤 AMI 를 생성합니다.

```
sudo vi /etc/systemd/system/flask.service

[Unit]
After=network.target

[Service]
User=ubuntu
WorkingDirectory=/home/ubuntu
ExecStart=/home/ubuntu/venv/bin/python /home/ubuntu/app.py
Restart=always

[Install]
WantedBy=multi-user.target

sudo systemctl daemon-reload
sudo systemctl enable --now flask
systemctl status flask
```

정상 동작을 확인한 뒤 기존 WAS-A 를 기준으로 사용자 AMI 를 생성합니다. 예: **was-asg-v1**

#### AMI 에 포함되는 것

기준 WAS 의 Flask 소스코드(app.py), Python 가상환경/필요 패키지, systemd flask.service 설정을 포함한 상태에서 AMI 를 생성합니다. Internal ALB DNS 는 WAS 이미지에 넣지 않습니다. WAS 인스턴스는 ALB 뒤에서 Target 으로 동작하므로 ALB 주소를 알 필요가 없습니다.

### 4. Step 3 - Security Group 변경

| Security Group | Inbound  | Source     | 비고                 |
|----------------|----------|------------|--------------------|
| SG-ALB-WAS     | TCP 80   | SG-WEB     | WEB → Internal ALB |
| SG-WAS         | TCP 5000 | SG-ALB-WAS | ALB → Flask        |
| SG-WAS         | TCP 22   | SG-BASTION | 관리 접속 유지           |
| SG-RDS         | TCP 3306 | SG-WAS     | 기존 설정 유지           |

#### 전환 순서

처음에는 기존 SG-WEB → SG-WAS :5000 규칙을 잠시 유지합니다. Internal ALB 와 ASG 가 정상 동작하고 Nginx 를 ALB DNS 로 변경한 뒤, 기존 WEB → WAS 직접 허용 규칙을 삭제합니다. 최종 상태에서는 WEB 이 WAS Private IP 의 :5000 으로 직접 접속할 수 없어야 합니다.

#### 최종 통신 정책

```
WEB -----> Internal ALB :80      허용
Internal ALB -> WAS :5000          허용
WAS -----> RDS :3306             허용
Bastion ----> WAS :22              허용

WEB -----X WAS Private IP :5000   차단
```

## 5. Step 4 - Target Group 과 Internal ALB

### 5.1 Target Group - TG-WAS

| 항목                | 설정          |
|-------------------|-------------|
| Target type       | Instances   |
| Protocol / Port   | HTTP / 5000 |
| VPC               | 기존 VPC      |
| Health check path | /health     |
| Success code      | 200         |

### 5.2 Internal Application Load Balancer

| 항목              | 설정                                      |
|-----------------|-----------------------------------------|
| Name            | alb-was-internal                        |
| Scheme          | Internal                                |
| IP address type | IPv4                                    |
| Subnet          | WAS-A 10.0.11.0/24 + WAS-C 10.0.21.0/24 |
| Security Group  | SG-ALB-WAS                              |
| Listener        | HTTP :80 → TG-WAS                       |

#### 중요

Internal ALB 는 인터넷 공개용이 아닙니다. WEB 서버가 VPC 내부에서 ALB DNS 로 요청하고, ALB 가 Healthy 상태의 WAS 인스턴스로 분산합니다.

## 6. Step 5 - Launch Template 과 Auto Scaling Group

### 6.1 Launch Template

| 항목             | 권장 설정          |
|----------------|----------------|
| AMI            | was-asg-v1     |
| Instance type  | t3.micro       |
| Key pair       | 기존 실습 Key Pair |
| Security Group | SG-WAS         |
| Public IP      | 할당하지 않음        |

### 6.2 Auto Scaling Group - ASG-WAS

| 항목                     | 설정                           |
|------------------------|------------------------------|
| VPC/Subnet             | WAS-A + WAS-C                |
| Load balancing         | 기존 TG-WAS 연결                 |
| Health check           | EC2 + Elastic Load Balancing |
| Min capacity           | 1                            |
| Desired capacity       | 1                            |
| Max capacity           | 3                            |
| Target tracking metric | Average CPU utilization      |
| Target value           | 30% 권장                       |
| Instance warmup        | 60 초 권장                      |

#### 실습 팁

CPU 기반 Auto Scaling 은 즉시 반응하지 않을 수 있습니다. CloudWatch 지표 수집과 인스턴스 준비 시간을 고려해 몇 분 간 지속적인 부하를 유지하십시오. Scale-Out 이 느리면 load=5 또는 동시 요청 수를 높여 확인합니다.

## 7. Step 6 - WEB Nginx 를 Internal ALB 로 전환

WEB-A 와 WEB-C 모두 개별 WAS IP 대신 Internal ALB DNS 를 바라보도록 변경합니다. 이 값은 ASG 인스턴스 수와 무관하며 ALB 를 재생성하지 않는 한 Nginx 설정을 다시 바꿀 필요가 없습니다.

```
location / {
    proxy_pass http://INTERNAL_ALB_DNS;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
}
```

```
sudo nginx -t
sudo systemctl restart nginx
```



## ALB DNS 주소 관리

Internal ALB 를 생성하면 AWS 가 고유한 DNS Name 을 제공합니다. WEB-A 와 WEB-C 의 Nginx proxy\_pass 에 이 DNS Name 을 한 번 입력하면 됩니다. WAS 가 1 대에서 2 대, 3 대로 Scale-Out/Scale-In 되어도 ALB DNS Name 은 바뀌지 않으며 Target Group 이 새 WAS 를 자동 등록/해제합니다. ALB 의 개별 IP 주소는 변경될 수 있으므로 IP 를 직접 입력하지 않습니다. ALB 자체를 삭제하고 새로 만들었을 때만 새 DNS Name 으로 Nginx 설정을 갱신합니다. 운영 환경에서는 Route 53 Private Hosted Zone 의 Alias 레코드로 고정된 내부 이름(예: was.internal.example.com)을 사용할 수 있습니다.

## Cutover 완료 후

서비스가 정상이라면 기존 SG-WEB → SG-WAS :5000 직접 허용 규칙을 삭제합니다. 이 시점부터 WEB 은 ALB 를 통해서만 WAS 에 접근해야 합니다.

## 8. Step 7 - Stress 서버 구축

| 항목            | 설정                    |
|---------------|-----------------------|
| 위치            | 기존 Public Subnet 중 하나 |
| Instance type | t3.micro              |
| Public IP     | 0                     |
| Inbound       | SSH :22 ← 관리자 IP      |
| Outbound      | 기본 허용                 |

```
sudo apt update
sudo apt install -y apache2-utils curl
```

## 9. Step 8 - Auto Scaling 테스트

### 9.1 부하 전 정상 상태 확인

```
for i in {1..10}; do
  curl -s "http://WEB_PUBLIC_IP/stress?load=1"
done
```

초기에는 ASG Desired=1 이므로 같은 WAS hostname 이 주로 확인됩니다.

### 9.2 지속적인 부하 발생

```
while true; do
  ab -n 1000 -c 20 "http://WEB_PUBLIC_IP/stress?load=3" > /dev/null
done
```

종료는 Ctrl+C. Scale-Out 이 충분히 발생하지 않으면 load=5 또는 -c 30 정도로 한 단계 높입니다.

### 9.3 관찰할 항목

- ☐ CloudWatch 에서 ASG 평균 CPU 가 Target Value 를 초과하는지 확인
- ☐ Auto Scaling Activity 에서 새 EC2 Launch 기록 확인

- Desired Capacity 가 1 → 2 → 3 으로 증가하는지 확인
- tg-was 에 새 Target 이 자동 등록되는지 확인
- 새 Target 이 Initial → Healthy 로 변경되는지 확인

## 9.4 Scale-Out 후 실제 분산 확인

```
for i in {1..20}; do
  curl -s "http://WEB_PUBLIC_IP/stress?load=1"
done
```

예상 출력 예

```
WAS Server: ip-10-0-11-53 / load=1
WAS Server: ip-10-0-21-87 / load=1
WAS Server: ip-10-0-11-142 / load=1
WAS Server: ip-10-0-21-87 / load=1
...
```

## 10. Step 9 - Scale-In 및 통신 정책 검증

Stress 를 Ctrl+C 로 중단한 뒤 ASG Activity 와 Desired Capacity 를 관찰합니다. Target Tracking 정책에 따라 부하가 낮아지면 인스턴스가 순차적으로 종료됩니다.

### 10.1 반드시 성공해야 하는 통신

| Source       | Destination  | Port | 결과      |
|--------------|--------------|------|---------|
| WEB          | Internal ALB | 80   | SUCCESS |
| Internal ALB | WAS          | 5000 | SUCCESS |
| WAS          | RDS          | 3306 | SUCCESS |
| Bastion      | WAS          | 22   | SUCCESS |

### 10.2 반드시 실패해야 하는 통신

| Source   | Destination    | Port | 결과       |
|----------|----------------|------|----------|
| WEB      | WAS Private IP | 5000 | FAIL     |
| Internet | Internal ALB   | 80   | 직접 접근 불가 |
| Internet | WAS            | 5000 | 직접 접근 불가 |

```
# WEB 서버에서 직접 WAS 접근 차단 확인
nc -zv -w 3 WAS_PRIVATE_IP 5000
```

## 11. Step 10 - WAS v1 → v2 애플리케이션 업데이트

### 왜 인스턴스에 하나씩 접속해서 수정하면 안 되는가?

ASG 환경에서 실행 중인 WAS-1, WAS-2 만 SSH 로 직접 수정하면 이후 Scale-Out 으로 생성되는 WAS-3 는 Launch Template 의 기존 AMI 에서 다시 만들어져 버전이 달라질 수 있습니다. 따라서 애플리케이션 변경은 "소스 수정 → 새 AMI → Launch Template 새 버전 → Instance Refresh" 순서로 배포합니다.

### 11.1 v1 상태 확인

```
for i in {1..10}; do
  curl -s http://WEB_PUBLIC_IP/info
done

# 예상
WAS=ip-10-0-11-53 VERSION=v1
WAS=ip-10-0-21-87 VERSION=v1
```

배포 과정을 눈으로 확인하려면 Instance Refresh 직전에 Desired Capacity 를 2 로 유지하는 것을 권장합니다.

### 11.2 기존 WAS 에서 v2 준비

Bastion 을 통해 기존 WAS 에 접속하거나, 기존 AMI v1 으로 임시 Build EC2 를 하나 생성하여 app.py 의 버전만 변경합니다.

```
# app.py
APP_VERSION = "v2"

sudo systemctl restart flask
curl http://127.0.0.1:5000/info
```

정상 동작을 확인한 후 이 인스턴스로 새 AMI 를 생성합니다. 예: was-asg-v2

### 11.3 Launch Template Version 2 생성

#### Launch Template Version 2

기존 Launch Template 에서 Create new version 을 선택하고 AMI 만 was-asg-v2 로 변경합니다. Instance type, Key Pair, SG-WAS, Public IP 미할당 등 나머지 설정은 v1 과 동일하게 유지합니다. ASG 에서 Version 2 를 사용하도록 지정합니다.

### 11.4 Instance Refresh 실행

EC2 → Auto Scaling Groups → ASG-WAS → Instance refresh 에서 새 Launch Template Version 2 를 선택하여 Rolling 방식으로 교체를 시작합니다. 새 인스턴스가 Healthy 상태가 된 뒤 기존 v1 인스턴스가 순차적으로 교체되는지 확인합니다.

```
# 배포 중 반복 확인
for i in {1..30}; do
  curl -s http://WEB_PUBLIC_IP/info
  sleep 1
done
```

```
# Instance Refresh 진행 중 예
WAS=ip-10-0-11-53 VERSION=v1
WAS=ip-10-0-21-201 VERSION=v2
WAS=ip-10-0-11-53 VERSION=v1
WAS=ip-10-0-21-201 VERSION=v2
```

```
# Instance Refresh 완료 후
WAS=ip-10-0-11-192 VERSION=v2
WAS=ip-10-0-21-201 VERSION=v2
WAS=ip-10-0-11-192 VERSION=v2
```

Instance Refresh 가 완료된 후에는 어떤 WAS hostname 이 출력되더라도 VERSION=v2 여야 합니다. 이후 Scale-Out 되는 신규 WAS 도 Launch Template v2 를 사용하므로 v2 로 생성되어야 합니다.

## 11.5 웹 브라우저에서도 확인

```
WAS Server: ip-10-0-21-201 / Version: v2
```

```
상품 목록
1 / 노트북 / 1500000 원
2 / 키보드 / 100000 원
3 / 마우스 / 50000 원
4 / 모니터 / 300000 원
```

## 12. 수행평가 체크리스트

- ☐ 기존 3-Tier/RDS 환경을 재구축하지 않고 재사용함
- ☐ Flask /health 정상 응답 확인
- ☐ Flask /stress?load=1~5 정상 동작 확인
- ☐ 기존 WAS 의 Flask 자동 시작 구성 및 AMI 생성
- ☐ AMI 에는 WAS 코드/실행 환경만 포함하고 ALB DNS 는 WEB Nginx 에서 관리
- ☐ TG-WAS :5000 및 /health Health Check 구성
- ☐ Internal ALB 를 WAS-A/WAS-C Subnet 에 구성
- ☐ Launch Template 에 사용자 AMI, t3.micro, SG-WAS 적용
- ☐ ASG Min 1 / Desired 1 / Max 3 구성
- ☐ ASG 와 TG-WAS 연결 및 ELB Health Check 활성화
- ☐ WEB Nginx proxy\_pass 를 Internal ALB DNS 로 변경
- ☐ SG-WEB → WAS :5000 직접 허용 규칙 제거
- ☐ Stress 서버에서 HTTP 부하 발생
- ☐ WAS Scale-Out 확인
- ☐ 새 Target 자동 등록 및 Healthy 확인
- ☐ 여러 WAS hostname 으로 요청 분산 확인
- ☐ 부하 제거 후 Scale-In 확인
- ☐ WEB → WAS Private IP :5000 직접 접근 차단 확인

- ☐ Flask /info 에서 WAS hostname + VERSION=v1 확인
- ☐ 상품 화면에 WAS hostname + Version 표시 확인
- ☐ app.py 를 v2 로 변경하고 AMI was-asg-v2 생성
- ☐ Launch Template Version 2 생성 및 ASG 에 적용
- ☐ Instance Refresh 실행 및 v1/v2 교체 과정 확인
- ☐ Instance Refresh 완료 후 /info 반복 호출 결과가 모두 VERSION=v2 인지 확인
- ☐ 이후 Scale-Out 되는 신규 WAS 도 VERSION=v2 인지 확인

## 13. 평가표

| 평가 항목                                          | 배점  |
|------------------------------------------------|-----|
| WAS 코드 준비 (/health, /info, /stress, Version)   | 15  |
| Target Group + Internal ALB 구성                 | 15  |
| Launch Template + Auto Scaling Group           | 20  |
| Security Group 최소 권한 전환                        | 10  |
| Stress 부하 + Scale-Out / 분산 확인                  | 15  |
| Scale-In 및 차단 통신 검증                            | 5   |
| AMI v2 + Launch Template v2 + Instance Refresh | 15  |
| 웹서비스 /info 로 최종 VERSION=v2 검증                  | 5   |
| 총점                                             | 100 |

## 14. 최종 제출 화면

1. Internal ALB 의 Scheme, Subnet, Listener 화면
2. TG-WAS 의 Healthy Target 화면
3. ASG Min/Desired/Max 및 Scaling Policy 화면
4. Scale-Out 전후 EC2 수량 변화 화면
5. Auto Scaling Activity 의 Launch 기록
6. curl 반복 요청에서 서로 다른 WAS hostname 이 출력되는 화면
7. Stress 종료 후 Scale-In 화면
8. WEB → WAS Private IP :5000 접속 실패 화면
9. 기존 상품 조회 서비스가 정상 동작하는 최종 브라우저 화면
10. /info 반복 호출에서 여러 WAS hostname 과 VERSION=v1 이 확인되는 화면
11. Launch Template Version 2 와 새 AMI v2 설정 화면
12. Instance Refresh 진행 화면 및 v1/v2 혼재 응답 화면
13. Instance Refresh 완료 후 /info 응답이 모두 VERSION=v2 인 화면
14. 최종 상품 화면 상단에 WAS hostname / Version v2 가 표시된 화면

## Mission Complete

```

기존
WEB ---> WAS Private IP ---> RDS

Mission 3 - Scale Out
WEB ---> Internal ALB ---> WAS Auto Scaling Group ---> RDS
                                |
                                WAS-1 / WAS-2 / WAS-3

Mission 3 - Application Update
AMI v1 ---> Launch Template v1 ---> WAS VERSION=v1
    |
    +--- app.py 수정 (APP_VERSION="v2")
        |
        v
        AMI v2 ---> Launch Template v2 ---> Instance Refresh
                                    |
                                    v
                                    모든 WAS VERSION=v2
  
```

### 참고 - AWS 공식 문서

Use an instance refresh to update instances in an Auto Scaling group -

<https://docs.aws.amazon.com/autoscaling/ec2/userguide/asg-instance-refresh.html>

Start an instance refresh using the AWS Management Console or AWS CLI -

<https://docs.aws.amazon.com/autoscaling/ec2/userguide/start-instance-refresh.html>

Attach an Elastic Load Balancing load balancer to your Auto Scaling group - <https://docs.aws.amazon.com/autoscaling/ec2/userguide/attach-load-balancer-asg.html>

Target tracking scaling policies for Amazon EC2 Auto Scaling - <https://docs.aws.amazon.com/autoscaling/ec2/userguide/as-scaling-target-tracking.html>

Tutorial: Set up a scaled and load-balanced application - <https://docs.aws.amazon.com/autoscaling/ec2/userguide/tutorial-ec2-auto-scaling-load-balancer.html>

Application Load Balancers - DNS name -

<https://docs.aws.amazon.com/elasticloadbalancing/latest/application/application-load-balancers.html>

Route 53 alias to ELB - <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/routing-to-elb-load-balancer.html>